

Harmony v3

Manual de inicio local

Go 1.22 + Gin

React 19 + Vite + TypeScript

PostgreSQL 15 + Redis

¿Qué es este manual?

Este documento explica paso a paso cómo levantar el entorno de desarrollo local de Harmony v3. El sistema está dividido en dos procesos independientes: el **backend Go/Gin** (puerto 8080) y el **frontend React/Vite** (puerto 3001). Ambos deben correr simultáneamente en terminales separadas.

La ruta base del proyecto en esta máquina es:

```
C:\Users\jarri\OneDrive\Documentos\Proyectos\Harmony v3\
```

■ Siempre trabaja desde esta ruta. Los comandos a continuación asumen que esta carpeta existe y que el repositorio ya fue clonado.

1. Requisitos previos

Herramienta	Version minima	Verificar con
Go	1.22	go version
Node.js	18 LTS	node --version
npm	9+	npm --version
PostgreSQL	15	psql --version
Redis	7	redis-cli ping

Instalacion en Windows

- **Go:** <https://go.dev/dl/> — descarga el instalador .msi
- **Node.js:** <https://nodejs.org> — descarga la version LTS
- **PostgreSQL:** <https://www.postgresql.org/download/windows/>
- **Redis:** Usar Memurai (<https://www.memurai.com>) o WSL2 con redis-server

■ Si usas Laragon, PostgreSQL y Redis ya pueden estar disponibles. Verifica en el panel de Laragon que ambos servicios esten activos.

2. Archivo de configuracion (.env)

El backend lee su configuracion de un archivo **.env** ubicado en la carpeta **api/**. Si no existe, crealo con el siguiente contenido:

RUTA DEL ARCHIVO:

```
C:\Users\jarri\OneDrive\Documentos\Proyectos\Harmony v3\api\.env
```

CONTENIDO DEL .ENV:

```
PORT=8080
APP_ENV=development
APP_KEY=harmony-dev-key-local

# PostgreSQL
DB_HOST=localhost
DB_PORT=5432
DB_USER=harmony
DB_PASS=harmony_secret
DB_NAME=harmony_system
DB_SSLMODE=disable

# Redis
REDIS_ADDR=localhost:6379
REDIS_PASSWORD=

# JWT
JWT_SECRET=cambia-este-secreto-en-produccion
JWT_EXPIRY_HOURS=24

# Frontend (CORS)
FRONTEND_URL=http://localhost:3001

# Opcional: Azure Blob Storage
AZURE_STORAGE_ACCOUNT=
AZURE_STORAGE_KEY=
AZURE_STORAGE_CONNECTION_STRING=

# Opcional: Anthropic (Bot IA)
ANTHROPIC_API_KEY=sk-ant-...
```

■ Si el usuario o contraseña de PostgreSQL son diferentes en tu instalacion, ajusta DB_USER y DB_PASS. El nombre de la base de datos del sistema siempre debe ser harmony_system.

3. Preparar la base de datos

Harmony v3 usa un esquema multi-tenant: una base de datos central **harmony_system** y una por empresa (**harmony_c1**, **harmony_c2**, etc). El primer paso es crear la DB central.

1 Abrir psql

Abre una terminal PowerShell y conecate a PostgreSQL.

```
psql -U postgres
```

2 Crear usuario y base de datos

Ejecuta estos comandos dentro de psql:

```
CREATE USER harmony WITH PASSWORD 'harmony_secret';  
CREATE DATABASE harmony_system OWNER harmony;  
\q
```

3 Las migraciones del sistema se ejecutan automaticamente

Cuando el servidor Go arranca por primera vez, corre las migraciones SQL ubicadas en `api/migrations/system/` automaticamente. No hace falta ejecutarlas manualmente.

■ Si quieres crear una empresa de prueba manualmente, el endpoint es `POST /api/admin/companies`. Al crearse, se provisiona automaticamente la base de datos `harmony_c{id}` con todas sus tablas.

4. Iniciar el backend (Go)

El servidor Go corre en el puerto 8080. Debes ejecutarlo desde la carpeta **api/** del proyecto.

1 Abrir terminal PowerShell

Abre una nueva ventana de PowerShell (o pestaña de Windows Terminal).

2 Navegar a la carpeta del API

Cambia el directorio a la carpeta **api/** del proyecto.

```
cd "C:\Users\jarri\OneDrive\Documentos\Proyectos\Harmony v3\api"
```

3 Ejecutar el servidor

Inicia el servidor con el comando **go run**. Veras logs en la consola.

```
go run cmd/server/main.go
```

SALIDA ESPERADA AL INICIAR:

```
2024/01/15 10:00:00 Corriendo migraciones del sistema...
2024/01/15 10:00:00 v Conectado a harmony_system
2024/01/15 10:00:00 v WebSocket Hub iniciado
2024/01/15 10:00:00 v Servidor corriendo en http://localhost:8080
[GIN-debug] GET    /api/auth/me
```

■ Deja esta terminal abierta. Si la cierras, el servidor se detiene. Los errores de base de datos o de conexión aparecen aquí.

Como reiniciar el servidor Go

Cuando modificas archivos **.go** debes detener el proceso y volver a iniciarlo. Usa este script de PowerShell:

```
# Detener el proceso en el puerto 8080
$p = (Get-NetTCPConnection -LocalPort 8080 -State Listen `
    -ErrorAction SilentlyContinue).OwningProcess
if ($p) { Stop-Process -Id $p -Force; Write-Host 'Servidor detenido' }

# Reiniciar
Start-Process -FilePath "C:\Program Files\Go\bin\go.exe" `
    -ArgumentList "run", "cmd/server/main.go" `
    -WorkingDirectory "C:\Users\jarri\OneDrive\Documentos\Proyectos\Harmony v3\api" `
    -WindowState Normal
```

■ Puedes guardar este script como **reiniciar_api.ps1** en la carpeta del proyecto para ejecutarlo rápidamente con un doble clic.

5. Iniciar el frontend (React / Vite)

El frontend corre en el puerto 3001 con Vite. Las llamadas a **/api/*** se redirigen automáticamente al backend en el puerto 8080 mediante el proxy configurado en vite.config.ts.

1 Abrir una segunda terminal

Abre otra ventana de PowerShell (mantén la del backend abierta).

2 Navegar a la carpeta web

Cambia el directorio a la carpeta web/ del proyecto.

```
cd "C:\Users\jarri\OneDrive\Documentos\Proyectos\Harmony v3\web"
```

3 Instalar dependencias (solo la primera vez)

Instala los paquetes npm. Solo necesario la primera vez o al actualizar package.json.

```
npm install
```

4 Iniciar el servidor de desarrollo

Arranca Vite en modo desarrollo. Hot Module Replacement (HMR) esta activo.

```
npm run dev
```

SALIDA ESPERADA:

```
VITE v6.x ready in 400 ms
```

```
-> Local: http://localhost:3001/
```

```
-> Network: http://192.168.x.x:3001/
```

■ También puedes usar el archivo start-dev.bat ubicado en la carpeta web/ haciendo doble clic. Ese archivo ejecuta npm run dev automáticamente.

6. Acceder al sistema

Recurso	URL	Notas
Frontend (App)	http://localhost:3001	Pagina principal
API backend	http://localhost:8080	Solo JSON, sin interfaz
Health check API	http://localhost:8080/api/auth/me	Requiere JWT en header

Iniciar sesion

Abre <http://localhost:3001> en el navegador. Ingresa con las credenciales del superadmin que se crean en las migraciones iniciales:

Campo	Valor por defecto
Email	admin@harmony.com
Contrasena	password

■ Si el login falla con 'credenciales invalidas', verifica que las migraciones se ejecutaron correctamente revisando los logs del backend en la primera terminal.

7. Resumen: flujo de inicio rapido

Cada vez que quieras trabajar con Harmony v3, sigue estos pasos en orden:

#	Terminal	Directorio	Comando
1	Terminal 1	...\api	go run cmd/server/main.go
2	Terminal 2	...\web	npm run dev
3	Navegador	—	http://localhost:3001

■ Para parar el sistema: Ctrl+C en cada terminal. Los datos en PostgreSQL persisten entre reinicios.

8. Solucion de problemas frecuentes

Error: 'port 8080 already in use'

Hay otro proceso usando el puerto 8080. Ejecuta en PowerShell:

```
$p = (Get-NetTCPConnection -LocalPort 8080 -State Listen).OwningProcess  
Stop-Process -Id $p -Force
```

Error: 'No se pudo conectar a harmony_system'

PostgreSQL no esta corriendo o las credenciales son incorrectas. Verifica:

```
# Verificar que PostgreSQL esta activo  
Get-Service -Name postgresql*  
  
# Probar conexion  
psql -U harmony -d harmony_system -c 'SELECT 1'
```

El frontend muestra errores de CORS o red

El backend no esta corriendo o FRONTEND_URL en .env no coincide. Verifica que go run esta activo en Terminal 1 y que .env tiene FRONTEND_URL=http://localhost:3001.

npm run dev falla con errores de modulos

Falta instalar dependencias. Ejecuta en la carpeta web/:

```
npm install
```

Login falla con 'invalid credentials'

Las migraciones no crearon el usuario inicial, o la DB esta vacia. Revisa los logs de go run en la primera terminal buscando errores de migracion.

9. Estructura del proyecto

Harmony v3/	
api/	Backend Go/Gin
cmd/server/main.go	Punto de entrada
internal/	Logica del sistema
config/	Variables de entorno
database/	Conexiones PostgreSQL
handlers/	Endpoints HTTP
models/	Structs GORM
router/	Definicion de rutas
ws/	WebSocket Hub
migrations/	SQL de migraciones
.env	Configuracion local
go.mod	
web/	Frontend React/Vite
src/	
api/	Cliente axios + hooks
components/	Sidebar, TopBar, etc.
pages/	Todas las vistas
hooks/	useAuth, useDarkMode
stores/	Zustand (auth)
index.html	
package.json	
vite.config.ts	

Puertos en uso

Puerto	Servicio	Descripcion
3001	Frontend Vite	Interfaz web (React)
8080	Backend Go	API REST + WebSocket
5432	PostgreSQL	Base de datos
6379	Redis	Cache y colas (Asynq)